

Data Availability Statement

For the manuscript: “What a radical-pair quantum reservoir would require: readable capacity, no quantum advantage, and a nuclear-register criterion”

Hikaru Wakaura and Taiki Tanimae, QIRI (Quantum Integrated Research Institute Inc.), Tokyo, Japan

h.wakaura@qiri.co.jp, t.tanimae@qiri.co.jp

Code. All results in this work are produced by the open-source Python package `qbsscreen` (MIT licence), available at https://github.com/deeptell-inc/new_Q_brain. The package implements the exact Liouville-space Haberkorn master equation with electronic decoherence and the correlated-pair-birth quantum-reservoir-computing protocol. The specific modules are:

- `qbsscreen/master_equation.py` — Liouville-space solver and its validation against the closed-form coherent limit (ESI, S1);
- `qbsscreen/reservoir.py` — reservoir Hamiltonian and propagator, and the out-of-sample information-processing capacity (IPC) and memory capacity (MC) estimators (trained on the first half of each run, scored on the held-out second half);
- `qbsscreen/corrected_injection.py` — the chemically consistent correlated-pair birth channel $\rho_e(s) = s|S\rangle\langle S| + (1-s)P_T/3$ and its S–T₀ coherent variant, together with the single-electron-reset control that reproduces the artifact it replaces;
- `qbsscreen/readout_routes.py` — the seven biologically accessible readout routes with recombination active, the product-weighted CIDNP observable, the turnover clock, and the nuclear-register depolarisation parameter q ;
- `qbsscreen/final_numbers.py` — protocol-consistent recomputation of the engineered-point capacity, coherence fraction, classical baselines and T_2^e scan; the chemical-readout, clock and criterion headline numbers come from `readout_routes.py` and `panel_response.py`;
- `qbsscreen/panel_response.py` — the referee-requested controls: the product-state register of Eq. (2), the per-route memoryless floors at $q = 1$, the delay-resolved memory kernel, the twelve-seed turnover-clock scan with paired statistics, the grid convergence of MC, the depolarising-versus-dephasing nuclear channel comparison, the cryptochrome-point classical baselines, and the ridge and shot-noise sensitivity scans. Outputs are written to `simulation_results/panel/`;
- `qbsscreen/qrc_benchmarks.py` — classical echo-state-network baselines, the NARMA2 benchmark, and the coherence-time scan;
- `qbsscreen/semiclassical.py` — the classical-spin reference model for the coherence fraction (ESI, S8);
- `qbsscreen/nuclide_register.py` — per-nucleus register relaxation, i.e. whether a proton-only register suffices (ESI, S9);
- `qbsscreen/ensemble_pooled.py` — pooled ensembles with heterogeneous couplings, desynchronised turnover, and heterogeneous relaxation times with the mean-field control that separates the spread from the shift in mean survival (ESI, S10);
- `qbsscreen/general_spin.py` — anisotropic hyperfine tensors and ¹⁴N as spin-1 (ESI, S11);
- `qbsscreen/relaxation_estimate.py` and `qbsscreen/premise_dependence.py` — every cryptochrome-point quantity rerun with the electron–electron couplings of Efimova and Hore (ESI Sec. S14); `qbsscreen/turnover_estimate.py` — prediction of the two constants the criterion depends on, with the relaxation calculator validated against three measured systems (ESI, S12);

- `qbscreen/product_carrier_audit.py` — the adverse sensitivity analyses re-run with the product register (ESI, S4);
- `manuscript/make_schematic.py` and `manuscript/make_fig_criterion.py` — Figs. 1–3.

Data. All numerical outputs underlying the figures and tables are provided as JSON files in `simulation_results/` of the repository, principally `final_numbers.json`, `corrected_injection.json`, `readout_routes.json`, `register_reuse.json`, `grid_convergence.json`, `qrc_benchmarks.json` and `cryptochrome_reality.json`, together with the referee-response outputs in `simulation_results/panel/` (`c1-c6`, `m6-m9`, `open1-open6`, `s2` and `audit3`), which carry the numbers added in revision. Three drivers—`ensemble.py`, `quantum_vs_classical.py` and `run_reservoir` in `reservoir.py`—still implement the superseded single-electron reset and are retained as controls only; no reported result derives from them, though the capacity estimators in `reservoir.py` are used throughout. No experimental data were generated; the study is entirely computational. Every reported capacity is out of sample, is quoted with its route-specific memoryless floor, and is shown to be converged in sample length and in the time grid; capacities are means over 3–12 input realisations with the stated standard deviations (the ridge and shot-noise sensitivity scans, the semiclassical integration-step check and the superseded clock scan of ESI Table S7 are single-realisation and are excluded): twelve for the turnover-clock scan, four for the register-reuse and nuclear-channel scans and the semiclassical reference, three for the ensemble and anisotropy scans, and six to eight elsewhere. The turnover-clock scan, which carries the central claim, uses twelve realisations; because the same seeds are used at every pause length, the change in capacity across pause lengths is reported as a paired difference with its standard error. An earlier version of this statement described the clock scan as using a single realisation, which was correct of the superseded scan and is no longer the case.

Reproducibility. The central claims—the agreement of the Liouville-space solver with the coherent limit, the Dambre capacity bound, the correlated-pair-birth readout capacities, and the nuclear-register reuse criterion—are covered by an automated test suite (`qbscreen/tests/`; 299 tests, all passing: `test_panel_claims.py` binds the headline quantities and load-bearing claims, and `test_table_rows.py` binds the capacity and standard-deviation cells of every table, with the expected values transcribed from the manuscript rather than from the data files, while `test_printed_cells.py` parses the printed cells back out of the `.tex` and checks them against the data, so a drift on either side fails. `test_spin_parameters.py` binds the input-parameter table and `test_cross_document_refs.py` the hand-typed cross-document table references. The tests that read the manuscript run on the $\text{\LaTeX}$ sources, which accompany this submission together with the freeze manifest rather than being distributed in the public repository; on a public clone those tests skip with that reason, and the collected count is unchanged. The table tests see only tables, so `test_prose_numbers.py` binds the criterion numbers printed in sentences—in this file’s companions `README.md`, the claims ledger and the cover letter as well as the manuscript—and fails when a new one appears with neither a binding nor a recorded waiver; derived-time and raw-IPC columns are not yet bound). Every number reported in the paper can be regenerated from a clean checkout with the commands below; `scripts/regenerate_all.sh` runs exactly this list and checks the result byte-for-byte against the shipped files; the three figures are rebuilt with `manuscript/make_schematic.py` and `manuscript/make_fig_criterion.py`. One shipped file, `open5_feasible_region.json`, duplicates a block inside `open5_turnover_estimate.json` and has no separate command.

```

pip install -e .
python -m pytest qbscreen/tests/ -q
python -m qbscreen.final_numbers
python -m qbscreen.readout_routes
python -m qbscreen.corrected_injection

```

```
python -m qbsscreen.reanalysis
python -m qbsscreen.panel_response
python -m qbsscreen.semiclassical
python -m qbsscreen.semiclassical floor
python -m qbsscreen.semiclassical conv
python -m qbsscreen.nuclide_register
python -m qbsscreen.ensemble_pooled
python -m qbsscreen.premise_dependence
python -m qbsscreen.general_spin
python -m qbsscreen.relaxation_estimate
python -m qbsscreen.turnover_estimate
python -m qbsscreen.product_carrier_audit
python -m qbsscreen.reservoir
python -m qbsscreen.reservoir realism
python -m qbsscreen.qrc_benchmarks
python -m qbsscreen.qrc_benchmarks tradeoff
python -m qbsscreen.qrc_benchmarks cryptochrome
python -m qbsscreen.ensemble
python -m qbsscreen.quantum_vs_classical
```

Software. Python 3, NumPy (≥ 1.24), SciPy (≥ 1.10) and Matplotlib (≥ 3.7). No proprietary software is required.

This statement follows the Royal Society of Chemistry Data Sharing Policy: <https://www.rsc.org/journals-books-databases/about-journals/data-sharing-policy/>